Proyecto Intermodular 2

Sprint 1 – Proyecto 1

Instrucciones breves de ejecución/despliegue



Tamara Morillo Gregorio 2º DAMS

Índice

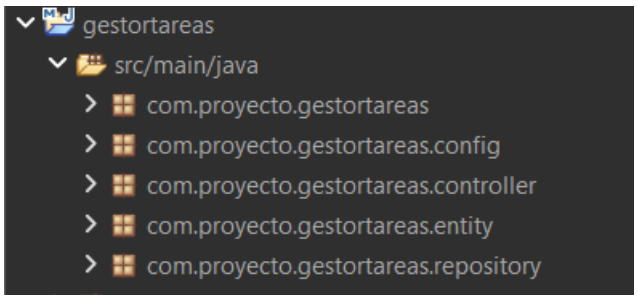
1. Entorno de desarrollo y estructura del proyecto 2
2. Creación y configuración de la base de datos 3
3. Conexión de Spring Boot con MySQL 4
4. Entidad usuario 5
5. Repositorio de usuarios 5
6. Controlador de usuarios 6
7. Registro de usuarios 7
8. Seguridad de las contraseñas 7
9. Inicio y cierre de sesión 8
10. Instrucciones de ejecución y despliegue 9

1. Entorno de desarrollo y estructura del proyecto

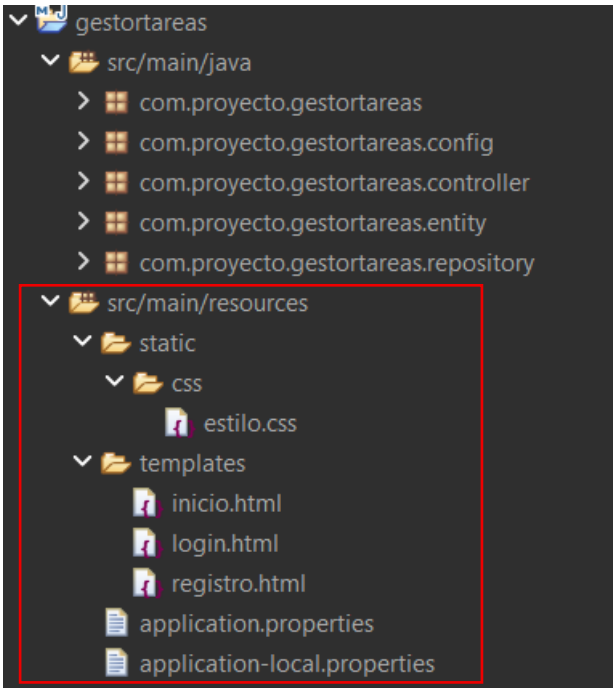
Para desarrollar **CatchTask** se ha utilizado como entorno de programación el IDE Eclipse, junto con Java 21, Spring Boot y Maven. Estas herramientas permiten desarrollar la aplicación web, organizar el código y gestionar las dependencias necesarias para su funcionamiento.

El proyecto se ha organizado en diferentes paquetes para separar las responsabilidades de cada parte de la aplicación:

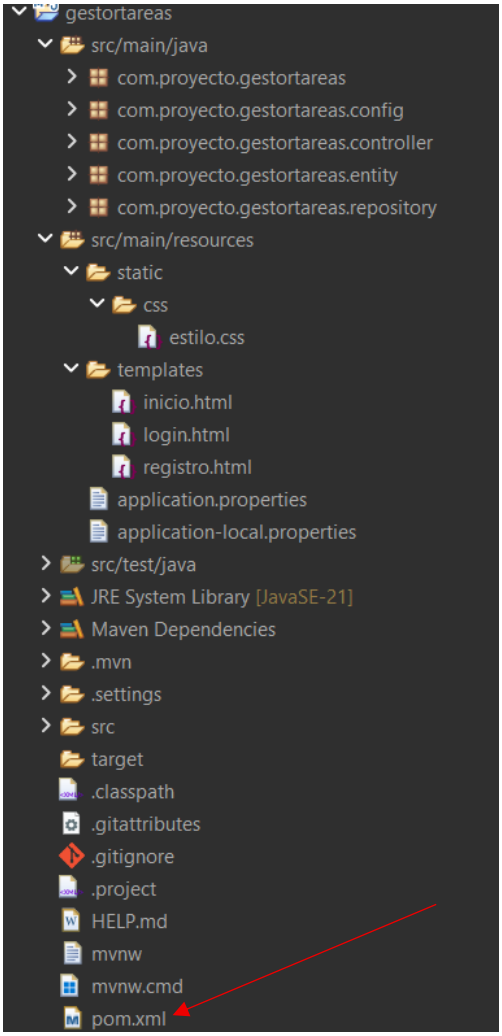
- **entity** contiene la clase Usuario, que representa los datos de los usuarios.
- **repository** permite acceder a la información almacenada en la base de datos.
- **controller** Gestiona las acciones que realiza el usuario en las páginas web, como registrarse, iniciar sesión y cerrar sesión.
- **config** contiene la configuración utilizada para proteger las contraseñas mediante BCrypt.



Además, dentro de **src/main/resources** se encuentran los recursos necesarios para el funcionamiento de la aplicación. Las páginas HTML se guardan en la carpeta **templates**, mientras que la hoja de estilos CSS se encuentra en **static/css/estilo.css**. Este archivo permite definir el diseño visual de las diferentes pantallas de **CatchTask**. En esta misma carpeta de recursos también se encuentra **application.properties**, donde se configura la conexión con la base de datos y otros aspectos de la aplicación.



Por último, el archivo **pom.xml** contiene la configuración de Maven y permite gestionar las dependencias necesarias para el funcionamiento del proyecto.



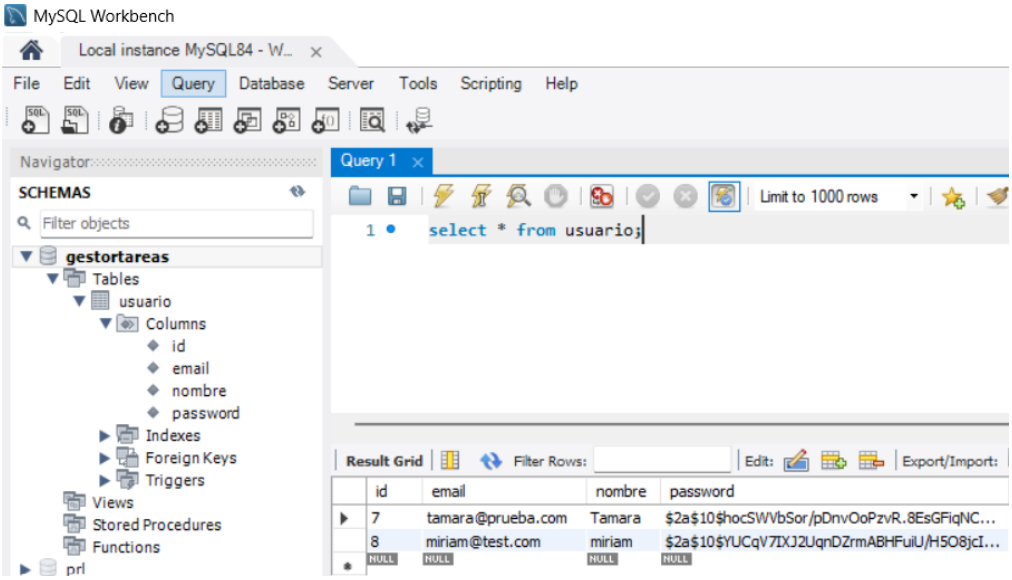
2. Creación y configuración de la base de datos

Para almacenar la información de los usuarios de **CatchTask**, se ha creado la base de datos **gestortareas** utilizando MySQL y se ha gestionado mediante MySQL Workbench.

Durante el Sprint 1 se ha creado la **tabla usuario**, que contiene los campos necesarios para registrar a los usuarios de la aplicación: **identificador, nombre, email y password**.

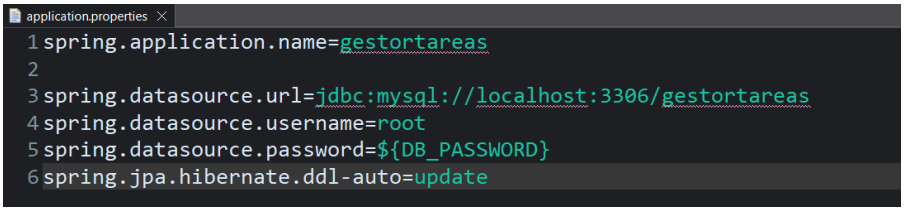
El campo id permite identificar de forma única a cada usuario, mientras que los campos nombre, email y password almacenan sus datos. Además, las contraseñas se guardan mediante un hash generado con BCrypt para mejorar la seguridad.

Esta base de datos permite almacenar la información de los usuarios y recuperarla cuando necesitan iniciar sesión en la aplicación.



3. Conexión de Spring Boot con MySQL

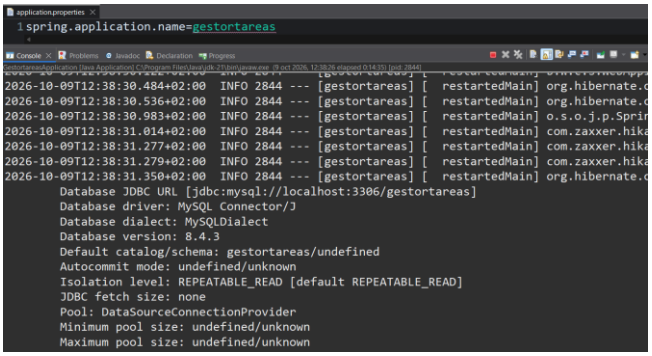
Para que **CatchTask** pueda guardar y consultar la información de los usuarios, se ha configurado la conexión entre Spring Boot y la base de datos MySQL en el archivo **application.properties**.



En este archivo se indica el **nombre de la aplicación**, la **dirección de la base de datos**, el **usuario de conexión** y la **configuración de JPA**. La contraseña de MySQL se obtiene mediante una variable de entorno, evitando escribirla directamente en el archivo de configuración. Esa contraseña se ha guardado en otro archivo llamado **application-local.properties.**

También se ha configurado **Hibernate** para actualizar la estructura de la base de datos cuando sea necesario. De esta forma, la aplicación puede trabajar con las tablas correspondientes a las entidades definidas en el proyecto.

Una vez configurada la conexión, se ha comprobado que la aplicación se inicia correctamente y puede almacenar y consultar los datos de los usuarios.



4. Entidad usuario

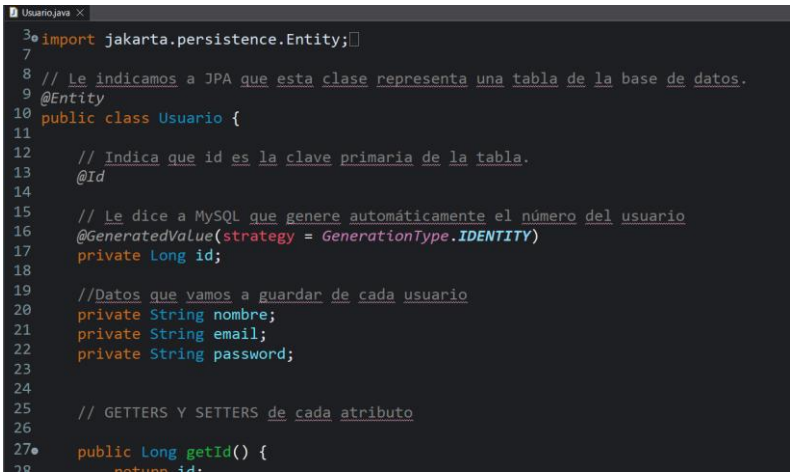
La clase **Usuario**, está situada en el paquete entity, que representa a los usuarios de **CatchTask** y define los datos que se almacenan en la base de datos.

Esta clase contiene los **atributos id, nombre, email y password**. El identificador id permite distinguir a cada usuario, mientras que el nombre, el correo electrónico y la contraseña almacenan la información necesaria para el registro y el inicio de sesión.

En esta clase se utilizan **tres anotaciones** principales:

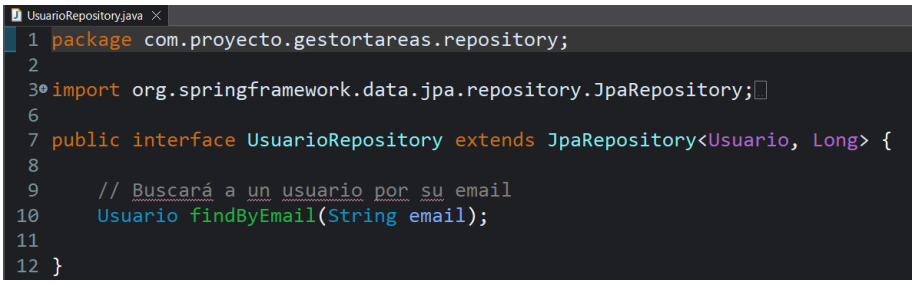
- **@Entity** indica que la clase representa una entidad que se relaciona con una tabla de la base de datos.
- **@Id** identifica el atributo que actúa como clave primaria, permitiendo distinguir a cada usuario.
- **@GeneratedValue** hace que el identificador de cada usuario se genere automáticamente, sin tener que escribirlo nosotros a mano.

Así, Hibernate conecta la clase Usuario con la tabla de MySQL y facilita guardar y consultar los datos de los usuarios.



5. Repositorio de usuarios

Esta **interfaz** permite acceder a los datos de los usuarios guardados en la base de datos. Gracias a Spring Data JPA, podemos **guardar y buscar** usuarios sin tener que escribir manualmente todas las consultas SQL. Este archivo sirve de conexión entre el código de la aplicación y los datos almacenados en MySQL.

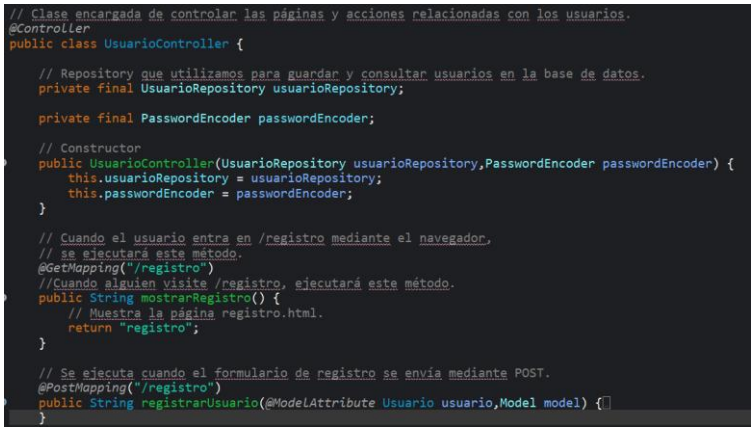


6. Controlador de usuarios

Esta clase se encarga de **gestionar** las **acciones que realiza el usuario en la aplicación**.

Este archivo **controla el registro, el inicio de sesión y el cierre de sesión**. También recibe los datos de los formularios y se comunica con el repositorio para consultar o guardar información en la base de datos. De esta forma, el controlador permite conectar las páginas HTML con el funcionamiento de la aplicación.

A continuación, se muestran dos capturas de pantalla mostrando parte del código:



```java
// Clase encargada de controlar las páginas y acciones relacionadas con los usuarios.
@Controller
public class UsuarioController {

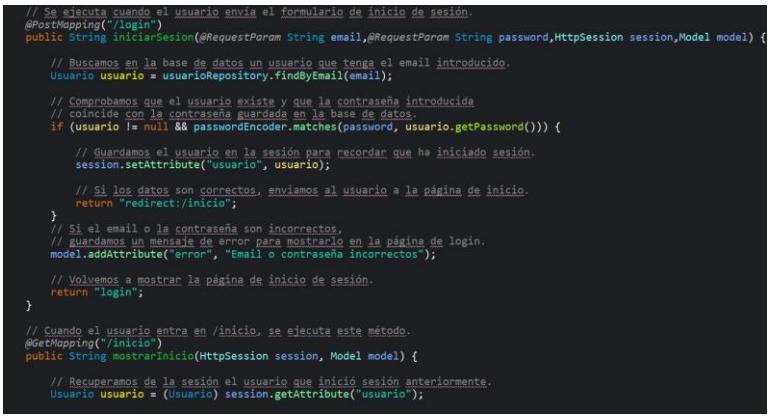
    // Repository que utilizamos para guardar y consultar usuarios en la base de datos.
    private final UsuarioRepository usuarioRepository;

    private final PasswordEncoder passwordEncoder;

    // Constructor
    public UsuarioController(UsuarioRepository usuarioRepository,PasswordEncoder passwordEncoder) {
        this.usuarioRepository = usuarioRepository;
        this.passwordEncoder = passwordEncoder;
    }

    // Cuando el usuario entra en /registro mediante el navegador,
    // se ejecutará este método.
    @GetMapping("/registro")
    //Cuando alguien visite /registro, ejecutará este método.
    public String mostrarRegistro() {
        // Muestra la página registro.html.
        return "registro";
    }

    // Se ejecuta cuando el formulario de registro se envía mediante POST.
    @PostMapping("/registro")
    public String registrarUsuario(@ModelAttribute Usuario usuario,Model model) {
    }
```



```java
// Se ejecuta cuando el usuario envía el formulario de inicio de sesión.
@PostMapping("/login")
public String iniciarSesion(@RequestParam String email,@RequestParam String password,HttpSession session,Model model) {

    // Buscamos en la base de datos un usuario que tenga el email introducido.
    Usuario usuario = usuarioRepository.findByEmail(email);

    // Comprobamos que el usuario existe y que la contraseña introducida
    // coincide con la contraseña guardada en la base de datos.
    if (usuario != null && passwordEncoder.matches(password, usuario.getPassword())) {

        // Guardamos el usuario en la sesión para recordar que ha iniciado sesión.
        session.setAttribute("usuario", usuario);

        // Si los datos son correctos, enviamos al usuario a la página de inicio.
        return "redirect:/inicio";
    }
    // Si el email o la contraseña son incorrectos,
    // guardamos un mensaje de error para mostrarlo en la página de login.
    model.addAttribute("error", "Email o contraseña incorrectos");

    // Volvemos a mostrar la página de inicio de sesión.
    return "login";
}

// Cuando el usuario entra en /inicio, se ejecuta este método.
@GetMapping("/inicio")
public String mostrarInicio(HttpSession session, Model model) {

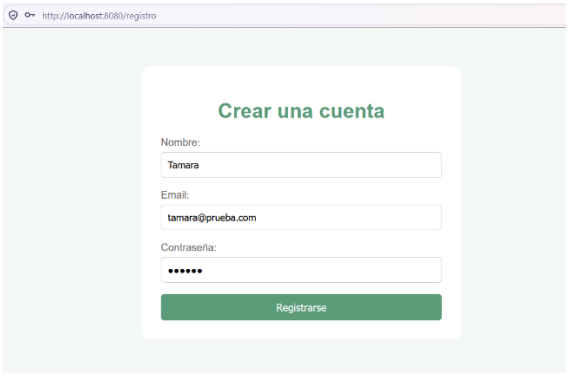
    // Recuperamos de la sesión el usuario que inició sesión anteriormente.
    Usuario usuario = (Usuario) session.getAttribute("usuario");
```

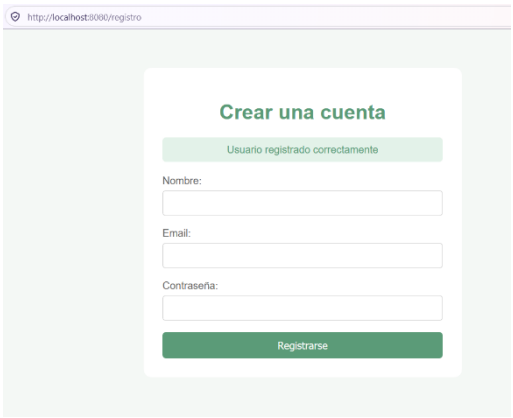
7. Registro de usuarios

La aplicación dispone de un formulario que permite a los nuevos usuarios registrarse introduciendo su nombre, correo electrónico y contraseña.

Cuando se envía el formulario, la aplicación procesa los datos y guarda la información del usuario en la base de datos MySQL. Antes de almacenar la contraseña, se aplica un sistema de cifrado mediante BCrypt para protegerla.

Si el registro se realiza correctamente, se muestra un mensaje de confirmación al usuario. De esta forma, se comprueba que el formulario funciona y que los datos se guardan correctamente.





8. Seguridad de las contraseñas

Para proteger las contraseñas de los usuarios, **CatchTask** utiliza BCrypt. Este sistema genera un hash de la contraseña antes de guardarla en la base de datos.

De esta forma, las contraseñas no se almacenan directamente en texto plano. Cuando un usuario inicia sesión, la aplicación comprueba si la contraseña introducida coincide con el hash guardado.

Con esto, mejoramos la seguridad de los datos de los usuarios.

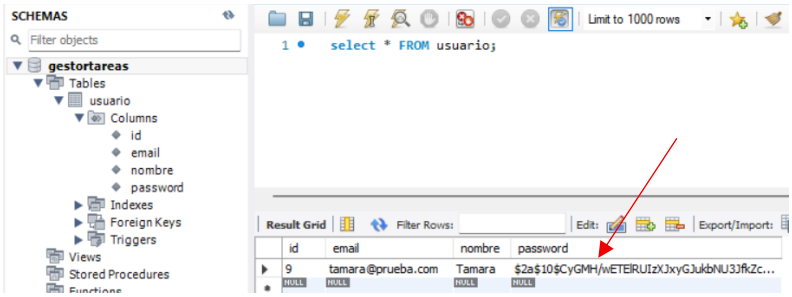
```java
package com.proyecto.gestortareas.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
public class PasswordConfig {

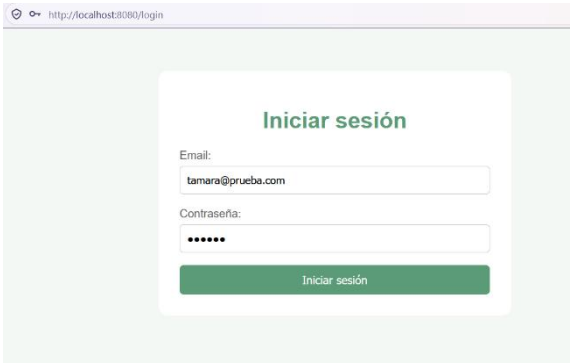
    // Crea el componente que utilizaremos para proteger las contraseñas
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

}
```

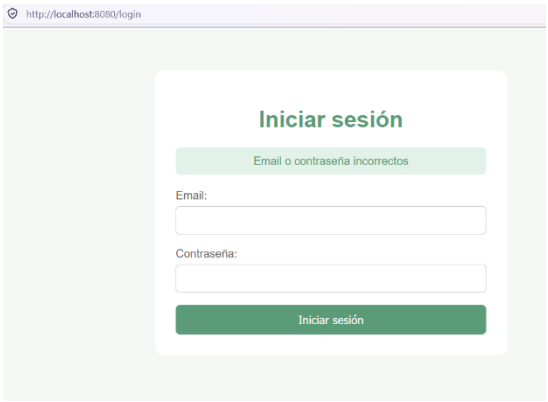


9. Inicio y cierre de sesión

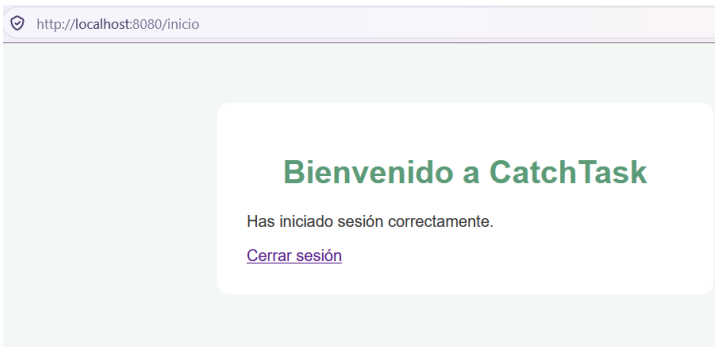
CatchTask permite a los usuarios iniciar sesión introduciendo su correo electrónico y contraseña. La aplicación comprueba los datos introducidos y, si son correctos, permite acceder a la página principal.



Si los datos son incorrectos, se muestra un mensaje de error para informar al usuario.



Además, la aplicación permite cerrar sesión. Al hacerlo, se finaliza la sesión del usuario y se vuelve a la página de inicio de sesión. También se ha comprobado que no se puede acceder a la página principal sin haber iniciado sesión.



10. Instrucciones de ejecución y despliegue

Para ejecutar **CatchTask**, es necesario tener instalado Java 21, Eclipse y MySQL, además de configurar correctamente la conexión con la **base de datos gestortareas**.

1. **Iniciar MySQL:** abrir MySQL Workbench y comprobar que la conexión con el servidor MySQL funciona.

2. **Preparar la base de datos:** crear la base de datos gestortareas si todavía no existe.

3. **Abrir el proyecto:** importar o abrir el proyecto gestortareas en Eclipse.

4. **Configurar la contraseña:** en Eclipse, abrir las configuraciones de ejecución del proyecto (*Run Configurations*). En la pestaña *Environment*, añadir la variable DB_PASSWORD con la contraseña del usuario de MySQL.

5. **Ejecutar la aplicación:** abrir la clase principal del proyecto, llamada *GestortareasApplication.java* y pulsar *Run As - Java Application*.

6. **Acceder a la aplicación:** abrir el navegador y entrar en `http://localhost:8080/registro` para mostrar el formulario de registro.

Una vez realizados estos pasos, **CatchTask** podrá ejecutarse en local y guardar los datos de los usuarios en MySQL.